

CONFIDENTIAL · AUDIT REPORT

Breeding Planner Codebase Audit

Security vulnerabilities, logic bugs, compactness issues, UX findings, and top-priority improvements for the breeder application.

2 Critical

5 High

4 Medium

4 Low

DATE	AUDITED BY	BRANCH	SCOPE
July 7, 2026	Fable 5 (Anthropic)	staging/runtime-review-20260521	Full codebase

Table of Contents

Scope & Orientation

1 • Security Issues

[1.1 CRITICAL — Account takeover via password recovery](#)

[1.2 HIGH — Tokens stored in localStorage](#)

[1.3 MEDIUM — Missing CSRF/cookie contract in server/](#)

[1.4 LOW — Minor security notes](#)

2 • Logic Bugs

[2.1 HIGH — Deletions never propagate; records resurrect on sync](#)

[2.2 HIGH — Content-hash log keys duplicate entries after edits](#)

[2.3 MEDIUM — Transient fetch failure can overwrite cloud data](#)

[2.4 MEDIUM — Photos stripped before sync; never propagate across devices](#)

[2.5 LOW — Polygenic traits modelled incorrectly](#)

[2.6 LOW — Scheduling heuristic silently gives up](#)

3 • Compactness

[3.1 CRITICAL — App.jsx is a 23,847-line god file](#)

[3.2 CRITICAL — The whole app exists 2–4× in the repo](#)

[3.3 HIGH — Dead files and binaries tracked in git](#)

[3.4 MEDIUM — Dead client auth machinery against server/](#)

4 • Intuitiveness / UX

5 • Top High-Impact Improvements

6 • What's Actually Good

Findings Summary

ID	Category	Title	Severity
1.1	Security	Account takeover via password "recovery"	CRITICAL
1.2	Security	Auth + refresh tokens stored in localStorage	HIGH
1.3	Security	Missing CSRF/cookie contract in server/	MEDIUM
1.4	Security	CORS, body size, console info-leak	LOW
2.1	Logic	Deletions never propagate; records resurrect on sync	HIGH
2.2	Logic	Content-hash log keys duplicate entries after edits	HIGH
2.3	Logic	Transient fetch failure can overwrite cloud data	MEDIUM
2.4	Logic	Photos stripped before sync; never propagate across devices	MEDIUM
2.5	Logic	Polygenic traits modelled as single-locus dominant	LOW
2.6	Logic	Scheduling heuristic silently gives up after 120 iterations	LOW
3.1	Compactness	App.jsx is a 23,847-line god file (134 useState hooks)	CRITICAL
3.2	Compactness	Whole app duplicated 2–4× in the repo	CRITICAL
3.3	Compactness	Dead files + binaries tracked in git (~42k dead lines)	HIGH
3.4	Compactness	Dead client auth machinery against server/	MEDIUM

Scope & Orientation

The repository is unusually large and contains **multiple parallel copies of the same application**, which materially affects the audit findings throughout this report.

Surface	File / Path	Lines	Notes
Desktop/Web App	src/App.jsx	23,847	Active; routed via AuthShell.jsx
Mobile Capacitor Shell	src/features/mobile/MobileApp.jsx	2,918	Active
HTTP Client	src/shared/apiClient.ts	—	Shared layer
Backend A	server/	—	Express/Prisma; missing CSRF
Backend B	breeding-app-backend/	—	Express/Prisma; has CSRF + cookies
Railway Deploy Target	breeding-app-breeder/	24,571	Diverged duplicate of src/

Railway deploys `breeding-app-breeder/` — i.e. **not** the `src/` tree that most development happens in. This duality means fixes applied in `src/` may silently not reach production.

Security Issues

CRITICAL 1.1 — Account Takeover via Password "Recovery" authService

Affected files:

server/src/services/authService.ts:109-129

breeding-app-backend/src/services/authService.ts:235-260

The `recoverPassword` endpoint lets **anyone reset any user's password** by supplying only an `email` + `fullName` + a new password — no email loop, no one-time token, no proof of ownership:

```
if (!user || !user.isActive || normalizeFullName(user.fullName) !== normalizeFullName  
    throw new HttpError(404, ...);  
const passwordHash = await bcrypt.hash(input.newPassword, 12);  
await prisma.user.update({ where: { id: user.id }, data: { passwordHash, refreshToken
```

Both the email and full name of a breeder are effectively public (they appear on marketplace profiles and listings). The existing rate limit of 10 requests per 15 minutes provides no protection since no guessing is required — a single request is sufficient for a known target.

Recommendation: Remove this endpoint entirely. Replace with a standard emailed single-use expiring reset-token flow. Never accept a new password alongside identity claims in one unauthenticated request.

HIGH**1.2 — Auth + Refresh Tokens Stored in localStorage**

apiClient.ts

Affected file: src/shared/apiClient.ts:11–33, 77–99

Access tokens *and* long-lived 7-day refresh tokens are persisted in `localStorage` for all three scopes (breeder/lab/admin). Any XSS anywhere in the application results in full, refreshable session exfiltration. The deployed `breeding-app-backend` already issues `httpOnly` cookies + CSRF (`controllers/authController.ts:35–37`), so the secure path exists — the client simply doesn't use it.

Recommendation: Move to `httpOnly`-cookie + CSRF mode end-to-end. Stop persisting refresh tokens in JS-readable storage.

MEDIUM**1.3 — server/ Backend Missing
CSRF/Cookie Contract**

server/ vs breeding-app-backend/

Affected file: src/shared/apiClient.ts:267–300

The client calls `GET /auth/csrf-token` and runs a cookie-preferred auth state machine, but `server/src/routes/authRoutes.ts` has **no** CSRF route and `server/src/app.ts` never installs a cookie-parser or calls `res.cookie`. Against `server/`, this entire flow hits a 404 silently. Only `breeding-app-backend/` implements CSRF correctly, creating real ambiguity about which backend is authoritative.

Recommendation: Delete the stale backend; keep one source of truth (see §3.2).

LOW 1.4 — Minor Security Notes

`server/src/app.ts:37-44` — In non-production mode, CORS reflects **any** origin with `credentials:true`. Acceptable for dev but ensure `NODE_ENV=production` in all prod environments.

`express.json({ limit: "25mb" })` is a large unauthenticated-body DoS surface, compounded by snapshots that can carry large payloads.

`src/contexts/SharedBackendContext.jsx:65-68` — Logs backend/auth status to console in production builds (minor info leak).

✓ No `dangerouslySetInnerHTML`, `eval`, or `innerHTML` sinks were found anywhere in the codebase.

Logic Bugs

HIGH

2.1 — Deletions Never Propagate; Records Resurrect on Sync

breederDataService.ts

Affected file: server/src/services/breederDataService.ts:233–349

`upsertBreederSnapshot` only **creates/updates** animals, pairings, and clutches — there is **no delete path**. The client merge at `src/App.jsx:2850–2886` (`mergeRecordsByKey` / `mergeBreederSnapshots`) is a union of local + backend by key. When a user deletes an animal or pairing locally, the next `runCloudBreederSync` re-fetches it from the backend and merges it back in. **Deletions are effectively impossible across devices.**

Recommendation: Introduce soft-delete tombstones (`deletedAt`) that sync like other fields. Have the server hide/remove tombstoned rows. Filter tombstoned records out of all merge operations.

HIGH

2.2 — Content-Hash Log Keys Duplicate Entries After Edits

App.jsx:2648

Affected file: src/App.jsx:2648–2683

`getLogEntryKey` derives a record key from *content* (date/result/feed/notes...) when no explicit id exists. Editing any field of a feed, weight, or shed log entry changes its key, so `mergeLogArrays` treats the edited entry as a **new row** and keeps both the old and new versions — resulting in silent duplicate log entries after every sync round-trip.

The same pattern in `getSyncRecordKey` (:2616–2628) falls back to `name` / `label`, so renaming an animal that lacks a stable id can mis-merge or duplicate it.

Recommendation: Always assign and preserve stable UUIDs on log entries and records at creation. Never key merges on mutable content.

MEDIUM

2.3 — Transient Fetch Failure Can Overwrite Cloud Data

MobileApp.jsx:2154

Affected file: src/features/mobile/MobileApp.jsx:2154–2169`pushToCloud` does:

```
const latest = await fetchBreederSnapshot().catch(() => ({ animals: [], pairings: [] })
const merged = mergeMobileSnapshot(localData, latest);
await saveBreederSnapshot(merged);
```

If the GET fails transiently (offline blip, timeout), the code proceeds to **merge against an empty backend and then save**. Because other devices' newer edits are not in the local copy, this can overwrite or omit fields on the server.

Recommendation: Abort the save if the pre-merge fetch fails. Only proceed with merge-then-save on a confirmed successful read.

MEDIUM

2.4 — Photos Stripped Before Sync; Never Propagate Across Devices

App.jsx:2565

Affected file: src/App.jsx:2565–2597

`sanitizeBackendMediaValue` removes all embedded `data:` image strings before upload. Camera photos captured in the mobile app are stored as data URLs and are **silently discarded on upload**. A device that re-hydrates from the cloud loses all photos. The flagship "take photos on your phone" feature is effectively broken for multi-device setups.

Recommendation: Upload photos to object storage (S3, Cloudflare R2, etc.) and sync URLs, not base64 blobs. The sanitization function exists for good reason (payload size), but needs a proper media upload path before stripping.

LOW

2.5 — Polygenic Traits Modelled as Single-Locus Dominant

punnett.ts:160

Affected file: `src/genetics/punnett.ts:160`

Polygenic traits are handled identically to dominant single-locus genes. Polygenic traits don't Mendelian-segregate — this is genetically incorrect. The branch at line 160 also contains a dead code disjunct: `"co-dom"` is already handled by the preceding branch and is unreachable here.

Recommendation: Handle polygenic traits as a distinct category with appropriate UI labelling. Remove the unreachable `"co-dom"` branch.

LOW

2.6 — Scheduling Heuristic Silently Gives Up

App.jsx:4573

Affected file: `src/App.jsx:4573-4604`

`findNextAvailableDate` gives up after 120 iterations and silently returns a still-conflicting date. The male "rest" span is also a hardcoded 3-day block. Fine for typical loads but users receive no feedback when no valid slot was found.

Recommendation: Return a sentinel value (null / error) when no valid slot is found, and surface a visible warning to the user rather than silently using a conflicting date.

Compactness

CRITICAL 3.1 — App.jsx is a 23,847-Line God File `src/App.jsx`

Affected file: `src/App.jsx` (23,847 lines)

The main component `BreedingPlannerApp` (line 6,411) alone holds **134 `useState` hooks** (157 across the file). Dozens of modals (`HeatRackFormModal` , `TerrariumFormModal` , `RoomModal` , `FeedPrepModal` ...), all export/PDF/XLSX logic, all genetics-alias logic, and all sync/merge logic live in a single module. This is unmaintainable, untestable, and causes excessive re-renders because any state change triggers evaluation of the entire component tree.

Recommendation: Extract by domain into modules (`sync/` , `export/` , `spaces/` , `feed/` , `pairing/`). Move sub-components to their own files. Consolidate related state into reducers or Zustand stores. This is the single change that would do the most to improve maintainability, performance, and review velocity.

CRITICAL 3.2 — The Whole App Exists 2–4× in the Repository repo structure

Multiple diverged copies of the same application are committed side-by-side:

Duplicate	Lines	Notes
<code>src/App.jsx</code>	23,847	Active development tree
<code>breeding-app-breeder/src/App.jsx</code>	24,571	Railway deploy target — diverged
<code>server/</code>	—	Backend A: missing CSRF
<code>breeding-app-backend/</code>	—	Backend B: has CSRF — diverged

Additional surface folders: `breeding-app-admin`, `breeding-app-lab`, `breeding-app-marketplace`, `breeding-app-public`, `breeding-app-shared`, `apps/`, `packages/`, `shared/`.

The \$1.1 account-takeover vulnerability exists in **both** backends and must be fixed in multiple places.

Recommendation: Decide on a single source of truth per surface and delete the rest. Factor genuinely shared code into one package rather than copy-forking. Until this is resolved, any fix may silently not apply to the deployed version.

HIGH

3.3 — Dead Files and Binaries Tracked in Git (~42k Dead Lines)

repo
root

The following files are tracked in git but serve no purpose:

File	Lines	Problem
src/App.js	12,659	Imported nowhere; stale duplicate of App.jsx
HEAD_App.jsx	14,154	Old manual backup
main_App.jsx	14,253	Old manual backup
App_prev.jsx	14,154	Old manual backup
Breeding-planner-project.zip	—	Binary in git
node-v22.11.0-win-x64.zip	—	Binary in git (~70MB)
node-v22.11.0-win-x64/	—	Unpacked Node.js tree in git
apk-backups/*.apk	—	APK binaries in git

Recommendation: Delete the dead `.jsx` / `.js` copies. Remove binaries and zips from version control and add them to `.gitignore`. This will dramatically shrink clone sizes and eliminate the risk of accidentally editing the wrong file.

MEDIUM

3.4 — Dead Client Auth Machinery Against server/ `apiClient.ts`

Affected file: `src/shared/apiClient.ts:267-300`

The cookie-preferred + CSRF logic (see §1.3) is inert and silently broken when used against `server/`. These code paths execute but fail without error, making them dead weight that creates confusion during debugging.

Recommendation: Resolve the backend ambiguity (§3.2) first, then clean up the dead auth paths.

Intuitiveness / UX

Issue	Detail	Impact
Silent photo loss	Photos appear locally on mobile then vanish on desktop with no message, warning, or error of any kind (see §2.4)	Users lose trust in the app's data reliability without understanding why
Sync is conflict-blind	<code>runCloudBreederSync</code> reports "Synced N snakes" but gives no indication when a merge discarded or superseded local edits. Errors are transient toasts only.	Users can't tell whether their data actually made it to the cloud
Invisible offline queue failures	Offline queue auto-retries on <code>[online, queue.length]</code> but failures are swallowed to a toast with no persistent indicator of a stuck queue	A stuck queue is invisible; data appears to be saved but isn't
Misleading recovery error copy	"We couldn't verify that account" (§1.1) implies a safe, normal recovery flow that is actually a full account-takeover vector	Security-critical flow has misleading UI copy
Naming drift across copies	Parallel app copies mean a contributor can "fix" a bug in a file that isn't the one deployed (§3.2)	Developer confusion; fixes silently don't reach production

Top High-Impact Improvements

#	Improvement	Why It Matters
1	Kill the account-takeover flow (§1.1) + move to httpOnly cookies (§1.2)	Existential security risk. A single unauthenticated request can take over any breeder account. The secure cookie/CSRF path already exists in <code>breeding-app-backend</code> — it just needs to be wired up end-to-end and the insecure endpoint removed.
2	Add deletion tombstones + stable UUID keys to the sync protocol (§2.1, §2.2)	Without these, deletes are lossy and edits create duplicates across devices. This is the difference between a toy sync and a trustworthy one that users can rely on.
3	Collapse duplicate apps/backends to one source of truth (§3.2), then split App.jsx (§3.1)	Everything else — performance, testability, review velocity, ability to apply fixes correctly — is gated on resolving the duplication. App.jsx at 23k lines is a maintenance emergency.
4	Real photo sync via object storage (§2.4)	The flagship "take photos on your phone" feature silently doesn't work across devices. Upload to S3/R2 and sync URLs instead of stripping base64 blobs.
5	Purge dead files and binaries from git (§3.3)	~42,000 lines of stale JSX and large binary files are tracked. Removing them de-risks editing the wrong file and dramatically shrinks clone sizes.
6	Harden sync against partial failures (§2.3) + surface sync state visibly	Never write after a failed read. Make the offline queue, conflict status, and last-synced timestamp visible to users as persistent UI indicators, not transient toasts.

What's Actually Good

Security positives

- No `eval` , `dangerouslySetInnerHTML` , or `innerHTML` sinks found anywhere in the codebase
- Passwords bcrypt-hashed at cost 12 — correct and appropriate
- Refresh tokens are server-revocable (wiped on logout and password change)
- JWT secret is env-required and fails loudly at startup if missing (`server/src/config/env.ts:5`)
- Committed `.env.example` files carry only placeholders; real `server/.env` is git-ignored
- Auth routes are rate-limited; admin and marketplace routes enforce `requireRole`

Code quality positives

- Punnett cross distribution math (allele convolution, normalization) is sound — correct output aside from the polygenic modelling caveat in §2.5
- The mobile offline queue architecture (`pendingSync` + online event listener) is the right pattern — it just needs better error surfacing
- The httpOnly-cookie + CSRF infrastructure already exists in `breeding-app-backend` — the secure path is built, just not wired up on the client